

RSA Studio

Technical implementation report

Static browser-local RSA document signing and verification

Date: 2 October 2026 / Version: 1.0.0

1. Objective and delivered application

RSA Studio gives users a way to sign documents and inspect received signatures without uploading documents or private keys. Node.js and npm provide the build toolchain; the deployed application is a set of static HTML, JavaScript and CSS files. React and TypeScript implement the interface, Vite bundles the assets, Web Crypto performs RSA operations, and PKI.js/ASN1.js handle cryptographic containers.

The application provides detached signatures for arbitrary files, embedded CMS signatures for supported PDFs, RSA key generation/import/export, and self-signed or imported X.509 certificates. The interface supports Vietnamese and English plus light, dark and system themes. English documentation, this PDF report and an AI usage disclosure accompany the source.

The scope is a practical parameter-controlled browser application. It is not a FIPS-validated module, identity provider, qualified electronic-signature service or independently audited security product.

Delivered artifacts

- Production build: dist/; no Node.js process is needed on the production host.
- Application source, automated tests and manual GitHub Pages deployment workflow.
- README.md, report.pdf, AI_USAGE.md and reproducible report source/scripts.

Interface structure

The design uses neutral surfaces, a single blue accent and semantic result colors. A sidebar separates Sign, Verify and Keys. Each operation places document input beside key/profile information, followed by an inspectable verification record. Body/display system fonts and monospace key data avoid third-party font downloads.

The following is a schematic, not a captured screenshot. Real screenshots were not available because the sandbox denied browser launch.

```
Workspace Document + signature mode Key / profile
Sign File selection Public fingerprint
Verify Sign / verify action Certificate state
Keys -----
Verification record
```

Theme / VI-EN Document -> Signature -> Public key
Identity / coverage / certificate details

2. Architecture and data flow

The main thread owns interface state and selected files. A dedicated module worker owns the active cryptographic keys and certificate. A request/response service correlates worker messages. Heavy key generation, PEM encryption, CMS and PDF operations execute outside the UI thread; the PDF module loads only when needed.

```

Local file + UI selection
|
v
Worker request: generate / import / export / sign / verify
|
+--> Web Crypto: RSA-PSS, SHA-256, PBKDF2, AES
+--> PKI.js / ASN1.js: PKCS#8, X.509, CMS
+--> Bounded PDF structural adapter
|
v
Result + public metadata + local download URL

```

No backend endpoint, remote key store, account or cloud document service is involved. The worker receives local file bytes through structured cloning and returns result metadata or downloadable output. Private keys are exportable because the user explicitly requested local export. The active private CryptoKey remains in worker memory; exported PEM necessarily reaches the main thread temporarily for download.

Session handling

Clearing the session terminates the worker and removes selected files, key state, PEM/password inputs and generated download URLs. Reloading creates a fresh session. localStorage contains only rsa-theme and rsa-language. Exported files remain on the user's disk until the user removes them. JavaScript garbage collection does not provide guaranteed forensic memory zeroization.

Input and resource boundaries

The app processes one document at a time with a 50 MiB limit. PEM file input is bounded at 1 MiB and detached JSON at 16 KiB. Encrypted PEM import caps PBKDF2 work at two million iterations. The PDF adapter limits recursion, cross-reference history, object counts and decompressed stream size. Unsupported structures return errors rather than a successful verification result.

Static deployment

Relative asset URLs and hash navigation support root and subdirectory hosting. nginx serves dist/ under HTTPS. GitHub Pages uses a manual workflow with build and tests before artifact upload. HTTPS or localhost is required for the browser environment; direct file:// use is unsupported. Neither a live deployment nor a remote repository was created during implementation.

3. Cryptographic profile and FIPS boundary

FIPS 186-5 approves RSA signature schemes subject to requirements beyond algorithm selection, including key generation and randomness. Its RSA modulus minimum is 2048 bits, and RSA-PSS salt length is bounded by the hash output length. The standard and NIST's potential-errata notice are authoritative references [1].

RSA Studio generates 2048, 3072 or 4096-bit keys, with 3072 as the default and public exponent 65537. Imported keys use the same size allowlist and must have an odd exponent greater than 65536 and less than 2^{256} . Multiprime private keys are rejected. The app checks parameters it can inspect but does not attest browser prime-generation procedures or the underlying random-bit generator.

All new signatures use RSA-PSS, SHA-256, MGF1-SHA-256 and 32-byte salt. Web Crypto performs the padding, hashing and RSA primitive [2,3]. Exact original file bytes are supplied to the signing operation; text normalization and double hashing are avoided. The same key pair is intended for document signing, not encryption.

Keys and fingerprints

Public export uses SPKI PUBLIC KEY PEM. Private input/output uses PKCS#8 PRIVATE KEY or ENCRYPTED PRIVATE KEY PEM. Encrypted export uses PBES2/PBKDF2-HMAC-SHA-256, 600,000 iterations and AES-256-CBC, with independently generated salt and IV. The UI requires a password of at least 12 characters and matching confirmation. Plaintext export is available with a visible warning.

The fingerprint is SHA-256 of the DER SPKI public key. It identifies key bytes, not a person. Users must establish the relationship between that fingerprint and the claimed signer through an independent trusted channel.

Claims deliberately excluded

The application does not claim full FIPS 186-5 compliance, FIPS 140 validation, certified browser randomness, signer identity assurance, legal non-repudiation, or protection against compromised device/application code. Cryptographic signature correctness is reported separately from certificate trust, certificate dates and PDF revision coverage.

4. Signature formats and PDF behavior

Detached document signatures

The downloaded filename ends in `.signature.json`. The version-1 envelope includes algorithm, hash, mask function, salt length, Base64 signature and public-key fingerprint. It excludes document content and private keys. Verification requires the exact original file and an independently supplied public key or X.509 certificate.

```
version: 1
algorithm: RSA-PSS
hash: SHA-256
mgf: MGF1-SHA-256
saltLength: 32
fingerprint: SHA-256(SPKI), lowercase hexadecimal
signature: Base64 RSA signature
```

The cryptographic signature covers document bytes only. Envelope fingerprint metadata is not signed; a metadata mismatch is a separate warning. Unsupported versions/parameters and malformed encodings are rejected. A wrong key or changed document produces a signature mismatch.

Embedded PDF signatures

PDF signing appends a signature dictionary, field, updated AcroForm and catalog, plus an incremental cross-reference table. The SubFilter is `adbe.pkcs7.detached`. CMS SignedData contains the X.509 certificate, content-type and message-digest attributes, and RSA-PSS signature. PKI.js constructs and signs CMS [4,5]. Original page bytes are preserved.

ByteRange covers the output except the hexadecimal Contents signature container. Before returning a signed PDF, the application verifies its generated output. Verification follows the PDF object structure, checks exact container exclusion and range boundaries, then verifies CMS over the covered bytes. It does not locate signatures through simple text matching.

The bounded adapter supports classic cross-reference tables, Flate cross-reference streams with supported PNG predictors, and compressed object streams. It rejects hybrid references, XFA, unsupported filters and malformed structures. This version signs unsigned, unencrypted PDFs only. PDF verification supports SHA-256 RSA-PSS with matching MGF1 and salt lengths 0-32, plus SHA-256 PKCS#1 v1.5 for existing signatures.

A valid signature covering an earlier revision is displayed with an explicit change-after-signing warning. It does not authenticate appended bytes. This adapter does not perform complete DocMDP, rendering or PDF shadow-attack analysis.

Certificates and trust

Users import a matching single X.509 PEM certificate or create a one-year, non-CA self-signed

certificate with digitalSignature key usage. A certificate mismatch, invalid signing dates or restrictive key usage prevents signing. Embedded certificate subject and validity dates are shown during verification; identity trust remains unverified.

No trust-store/chain validation, OCSP/CRL, TSA or trusted timestamp is implemented. No PAdES compliance or long-term validation claim is made. Adobe Reader may label a self-signed signer untrusted; Adobe compatibility itself remains pending manual verification.

5. Validation evidence

Validation used Node.js 24.18.0, OpenSSL 3.6.2, Vite 8.0.16 and Vitest 4.1.8 on macOS. Dependency installation succeeded offline through available npm cache and a clean npm ci. Tests generate disposable keys locally; no production document or key was used.

Executed checks

- TypeScript checking and static production build.
- Core/PDF automated tests: signing/verification, all supported key sizes, weak-key rejection, PEM round trips, wrong passwords, certificate/key mismatch, PDF tampering and malformed ranges.
- PDF structural tests: compressed xref/object streams with PNG predictors, appended revisions, encryption/hybrid rejection and object generation-number checks.
- Independent OpenSSL tests: verify Web Crypto PSS, verify OpenSSL PSS in Web Crypto, encrypted PKCS#8 both directions, self-signed certificate verification and extracted PDF CMS verification.
- React UI integration tests: themes/languages, detached and PDF round trips, tampering, clearing sensitive state and oversized upload handling. These tests use jsdom with the actual production worker bundle over a Node worker transport.

The current suite contains 28 passing tests across four test files. The core and DOM integration checks establish behavior in this environment; jsdom is not a real browser renderer.

Checks blocked by the environment

Starting a localhost server failed with listen EPERM. Chromium startup separately failed because the managed sandbox denied macOS Mach-port registration. The Playwright suite recorded a blocked listener result instead of reporting success. No completed Chromium/Firefox/WebKit browser run, actual UI screenshot, responsive visual inspection, or Adobe Acrobat Reader check is claimed.

The provided Playwright suite covers generated/imported keys, downloads, detached/PDF signing and verification, tampering, theme changes, mobile overflow, temporary sessions, request inspection and subpath hosting. GitHub validation automation runs it in a normal CI environment after installing browser dependencies. Real browser and reader validation remains required before a high-assurance rollout.

Report artifact validation

The report is generated from this English Markdown source using a dependency-free paginated PDF writer. npm run report regenerates it. An optional --browser path uses Chromium printing and includes real screenshots when available. The report contains no fabricated test result or screenshot.

6. Implementation decisions and maintenance

The original PDF-library choice, `@libpdf/core`, was unavailable in the offline package cache. The delivered bounded structural adapter replaces that dependency while keeping RSA primitives in Web Crypto. Its supported subset is documented in the UI and README. This tradeoff makes parser review and regression coverage a maintenance responsibility; it should not be mistaken for a mature, comprehensive PDF validator.

The originally suggested bundled Manrope/Noto fonts were also unavailable. Operating-system display/body fonts and monospace utility text preserve a restrained design without runtime external requests. Browser restrictions necessitated the native report renderer instead of successful Playwright printing. Browser automation and the optional print path remain in the repository.

Recommended release checks are to run the prepared suite on the intended hosting origin and target browsers, inspect mobile/light/dark screenshots, open generated PDFs in the target reader, and review the PDF adapter and deployment code before handling sensitive documents. Certificate trust integrations, hardware tokens, encrypted PDF signing, additional signatures and timestamps require separate future work.

Reproduction commands

```
npm ci
npm run typecheck
npm test
npm run build
npx playwright install --with-deps chromium firefox webkit
npm run test:e2e
npm run report
```

AI-assisted work and verification boundaries are disclosed in `AI_USAGE.md`. No independent human audit, FIPS certification or live deployment is claimed.

References

- [1] NIST FIPS 186-5, Digital Signature Standard: <https://csrc.nist.gov/pubs/fips/186-5/final>
- [2] W3C Web Cryptography API: <https://www.w3.org/TR/webcrypto/>
- [3] RFC 8017, PKCS #1 v2.2: <https://www.rfc-editor.org/rfc/rfc8017>
- [4] RFC 5652, Cryptographic Message Syntax: <https://www.rfc-editor.org/rfc/rfc5652>
- [5] PKI.js SignedData API: <https://pkijs.org/docs/api/classes/SignedData/>
- [6] Vite static deployment: <https://vite.dev/guide/static-deploy.html>
- [7] Playwright PDF printing: <https://playwright.dev/docs/api/class-page#page-pdf>

